

Evaluate OWASP

Target : OWASP Juice shop

Compare between before testing and after testing

Before Testing:

- What kinds of Vulnerability exist in the web app

After testing:

Apply OWASP for testing, compare these aspects:

- + Check if we can find out all the vulnerabilities

- + Evaluate the Efficiency in context of time (slow, complex or fast, easy to execute)

-> Improve: How to make it more efficient (reduce time / complex and produce the same result)

I, Testing need to take

1, Information gathering

2, Authentication / Authorization testing

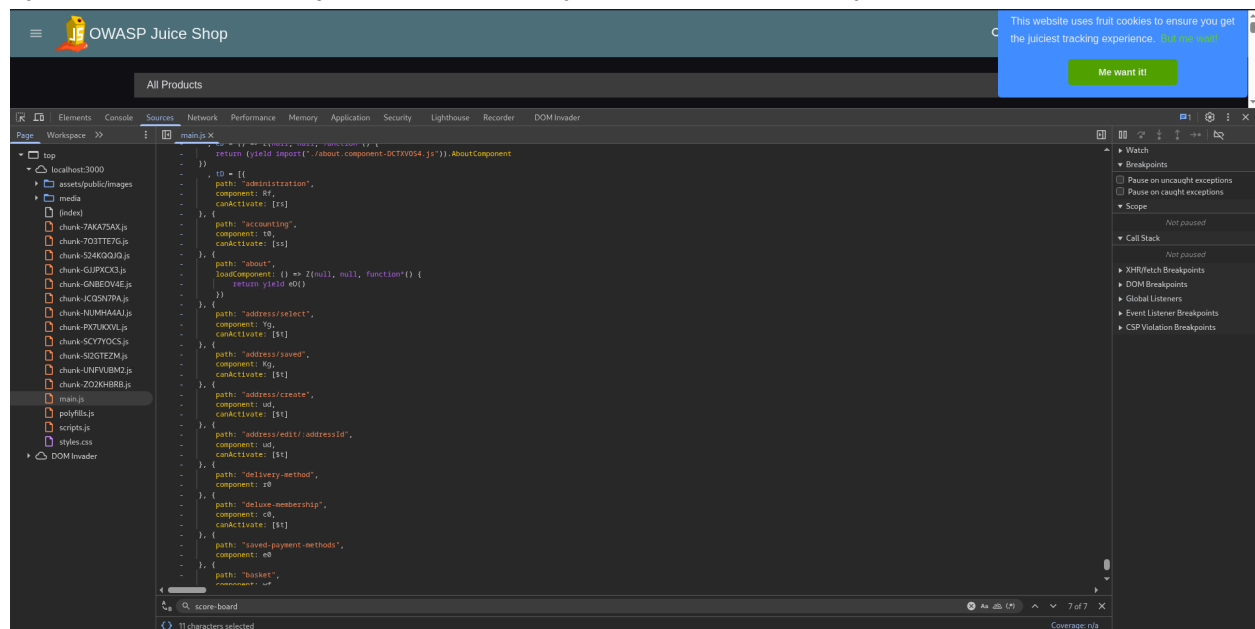
3, Input Validation Testing

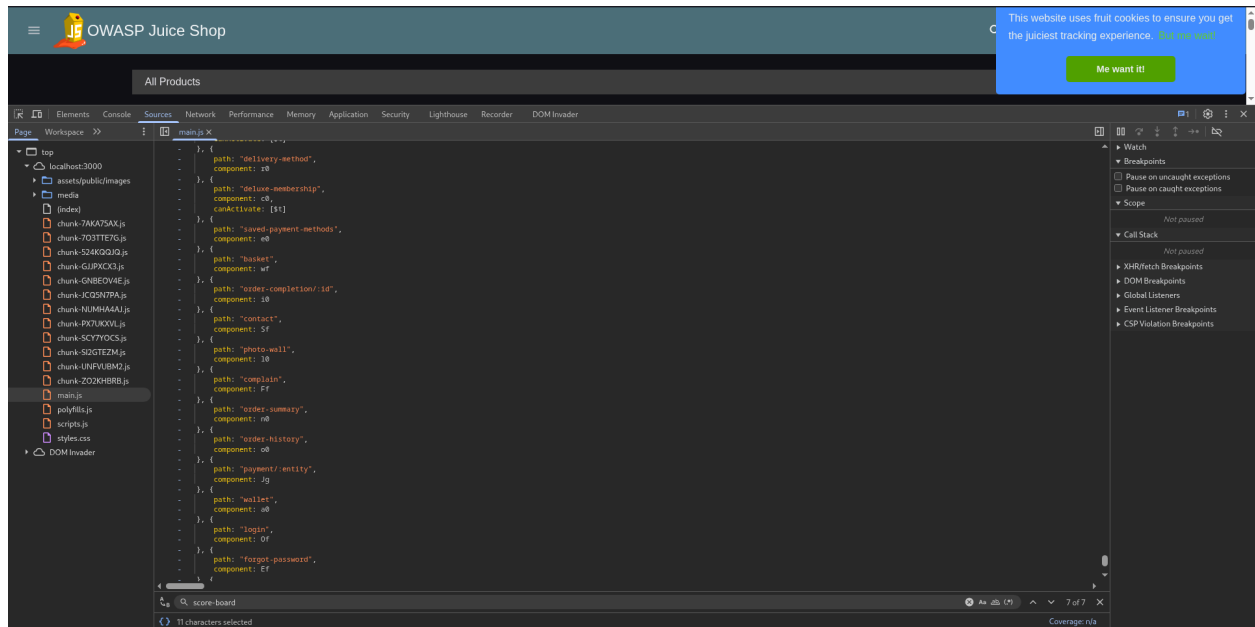
4, Business Logic Testing

TESTING

I, Information gathering

By inspect the html and js file, we can easily see the path directory of the web application





Detail:

```
, tD = {
  path: "administration",
  component: Rf,
  canActivate: [rs]
}, {
  path: "accounting",
  component: t0,
  canActivate: [ss]
}, {
  path: "about",
  loadComponent: () => Z(null, null, function*() {
    return yield eD()
  })
}, {
  path: "address/select",
  component: Yg,
  canActivate: [$t]
}, {
  path: "address/saved",
  component: Kg,
  canActivate: [$t]
}, {
  path: "address/create",
  component: ud,
  canActivate: [$t]
}, {
```

```
    path: "address/edit/:addressId",
    component: ud,
    canActivate: [$t]
  }, {
    path: "delivery-method",
    component: r0
  }, {
    path: "deluxe-membership",
    component: c0,
    canActivate: [$t]
  }, {
    path: "saved-payment-methods",
    component: e0
  }, {
    path: "basket",
    component: wf
  }, {
    path: "order-completion/:id",
    component: i0
  }, {
    path: "contact",
    component: Sf
  }, {
    path: "photo-wall",
    component: l0
  }, {
    path: "complain",
    component: Ff
  }, {
    path: "order-summary",
    component: n0
  }, {
    path: "order-history",
    component: o0
  }, {
    path: "payment/:entity",
    component: Jg
  }, {
    path: "wallet",
    component: a0
  }, {
    path: "login",
    component: Of
  }, {
```

```
    path: "forgot-password",
    component: Ef
  }, {
    path: "recycle",
    loadComponent: () => Z(null, null, function*() {
      return yield JA()
    })
  }, {
    path: "register",
    component: kf
  }, {
    path: "search",
    component: Bs
  }, {
    path: "hacking-instructor",
    component: Bs
  }, {
    path: "score-board",
    component: Pb
  }, {
    path: "track-result",
    component: jc
  }, {
    path: "track-result/new",
    component: jc,
    data: {
      type: "new"
    }
  }, {
    path: "2fa/enter",
    component: Nf
  }, {
    path: "privacy-security",
    component: zf,
    children: [{
      path: "privacy-policy",
      component: $g
    }, {
      path: "change-password",
      component: Lf
    }, {
      path: "two-factor-authentication",
      component: Gg
    }, {
```

```

    path: "data-export",
    component: Wg
  }, {
    path: "last-login-ip",
    component: Qg
  }]
}, {
  path: "juicy-nft",
  component: d0
}, {
  path: "wallet-web3",
  loadChildren: () => Z(null, null, function*() {
    return yield KA()
  })
}, {
  path: "web3-sandbox",
  loadChildren: () => Z(null, null, function*() {
    return yield XA()
  })
}, {
  path: "chatbot",
  component: Ob,
  children: [{
    path: "",
    component: Db
  }, {
    path: "conversation/:id",
    component: Bb
  }]
}, {
  path: "bee-haven",
  loadChildren: () => Z(null, null, function*() {
    return yield YA()
  })
})

```

By using crawling tool Burp Site we also know that there is a secret path /ftp that doesn't seem to appear in any HTML/JS source code

The screenshot shows a REST client interface with the following components:

- Contents:** A list of REST API endpoints with columns for Host, Method, URL, Params, Status code, Length, MIME type, Title, Notes, and Time requested.
- Issues:** A list of issues with columns for Host, Method, URL, Params, Status code, Length, MIME type, Title, Notes, and Time requested.
- Request:** A GET request to /rest/admin/application-configuration. The request body is empty.
- Response:** A 200 OK response with a JSON body containing configuration details for the OWASP Juice Shop application.

Additionally, by using some special letter to fill in log in form, the error return result indicates that the web server is using SQL :

The screenshot shows the OWASP Juice Shop login page. The login form has the following components:

- Email:** A text input field.
- Password:** A password input field containing a string of special characters.
- Login button:** A button labeled "Log in" that is disabled.
- Remember me:** A checkbox labeled "Remember me".
- Log in with Google:** A button labeled "Log in with Google".
- Not yet a customer?:** A link labeled "Not yet a customer?".

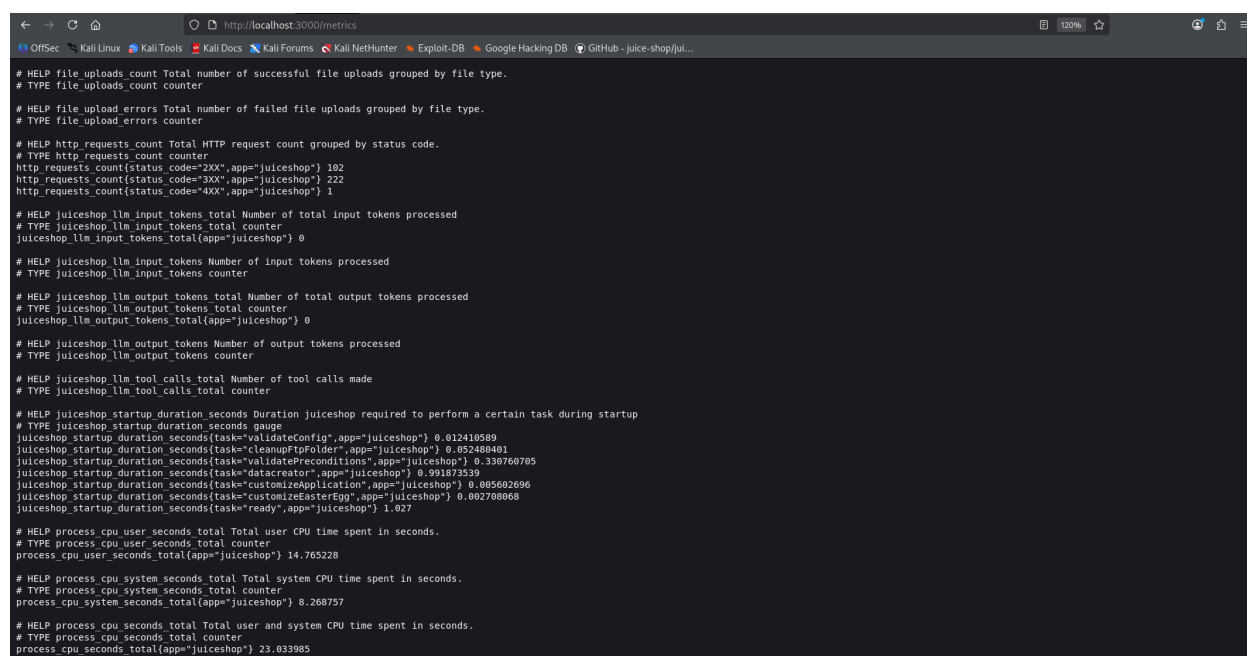
By trigger an error handling we can see some web application components like Angular, SQLite,...

OWASP Juice Shop (Express ^4.22.1)

500 Error: Unexpected path: /rest/qwert

```
at juice-shop/build/routes/angular.js:42:18
at juice-shop/build/lib/utils.js:225:26
at Layer.handle [as handle_request] (juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (juice-shop/node_modules/express/lib/router/index.js:328:13)
at juice-shop/node_modules/express/lib/router/index.js:286:9
at router.process_params (juice-shop/node_modules/express/lib/router/index.js:346:12)
at next (juice-shop/node_modules/express/lib/router/index.js:280:10)
at juice-shop/build/routes/verify.js:210:5
at Layer.handle [as handle_request] (juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (juice-shop/node_modules/express/lib/router/index.js:328:13)
at juice-shop/node_modules/express/lib/router/index.js:286:9
at router.process_params (juice-shop/node_modules/express/lib/router/index.js:346:12)
at next (juice-shop/node_modules/express/lib/router/index.js:280:10)
at juice-shop/build/routes/verify.js:113:5
at Layer.handle [as handle_request] (juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (juice-shop/node_modules/express/lib/router/index.js:328:13)
at juice-shop/node_modules/express/lib/router/index.js:286:9
at router.process_params (juice-shop/node_modules/express/lib/router/index.js:346:12)
at next (juice-shop/node_modules/express/lib/router/index.js:280:10)
at logger (juice-shop/node_modules/morgan/index.js:144:5)
at Layer.handle [as handle_request] (juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (juice-shop/node_modules/express/lib/router/index.js:328:13)
```

By access to /metrics , we can conclude that the web app use Prometheus for scraping usage data



```
# HELP file_uploads_count Total number of successful file uploads grouped by file type.
# TYPE file_uploads_count counter

# HELP file_upload_errors Total number of failed file uploads grouped by file type.
# TYPE file_upload_errors counter

# HELP http_requests_count Total HTTP request count grouped by status code.
# TYPE http_requests_count counter
http_requests_count{status_code="2XX",app="juiceshop"} 182
http_requests_count{status_code="3XX",app="juiceshop"} 222
http_requests_count{status_code="4XX",app="juiceshop"} 1

# HELP juiceshop_llm_input_tokens_total Number of total input tokens processed
# TYPE juiceshop_llm_input_tokens_total counter
juiceshop_llm_input_tokens_total{app="juiceshop"} 0

# HELP juiceshop_llm_input_tokens Number of input tokens processed
# TYPE juiceshop_llm_input_tokens counter

# HELP juiceshop_llm_output_tokens_total Number of total output tokens processed
# TYPE juiceshop_llm_output_tokens_total counter
juiceshop_llm_output_tokens_total{app="juiceshop"} 0

# HELP juiceshop_llm_output_tokens Number of output tokens processed
# TYPE juiceshop_llm_output_tokens counter

# HELP juiceshop_llm_tool_calls_total Number of tool calls made
# TYPE juiceshop_llm_tool_calls_total counter

# HELP juiceshop_startup_duration_seconds Duration juiceshop required to perform a certain task during startup
# TYPE juiceshop_startup_duration_seconds gauge
juiceshop_startup_duration_seconds{task="validateConfig",app="juiceshop"} 0.012410589
juiceshop_startup_duration_seconds{task="cleanupPtpFolder",app="juiceshop"} 0.052480491
juiceshop_startup_duration_seconds{task="validatePreconditions",app="juiceshop"} 0.330760705
juiceshop_startup_duration_seconds{task="datacreator",app="juiceshop"} 0.991873539
juiceshop_startup_duration_seconds{task="customizeApplication",app="juiceshop"} 0.005602696
juiceshop_startup_duration_seconds{task="customizeEasterEgg",app="juiceshop"} 0.002780068
juiceshop_startup_duration_seconds{task="ready",app="juiceshop"} 1.027

# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total{app="juiceshop"} 14.765228

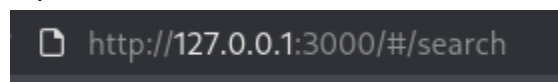
# HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE process_cpu_system_seconds_total counter
process_cpu_system_seconds_total{app="juiceshop"} 8.268757

# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total{app="juiceshop"} 23.033985
```

II, Authentication testing

1, Testing for Credentials transport over an Encrypted channel

The whole web application are using HTTP and cant switch to HTTPS. Request are transferred in plain text and can be sniffed.



2, Testing for Default Credentials

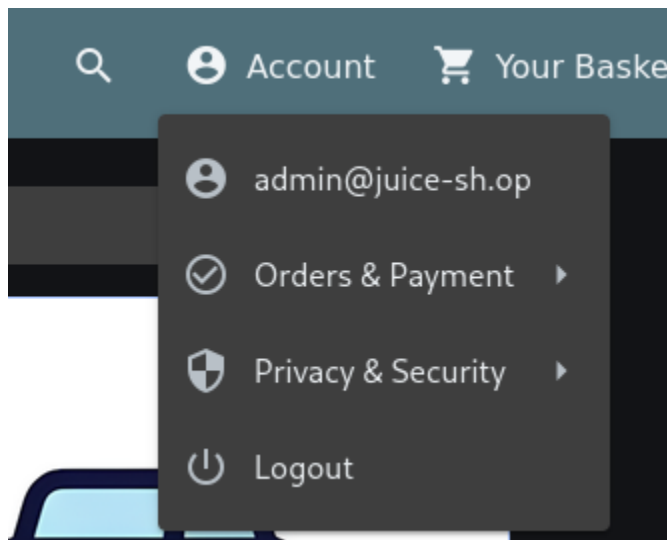
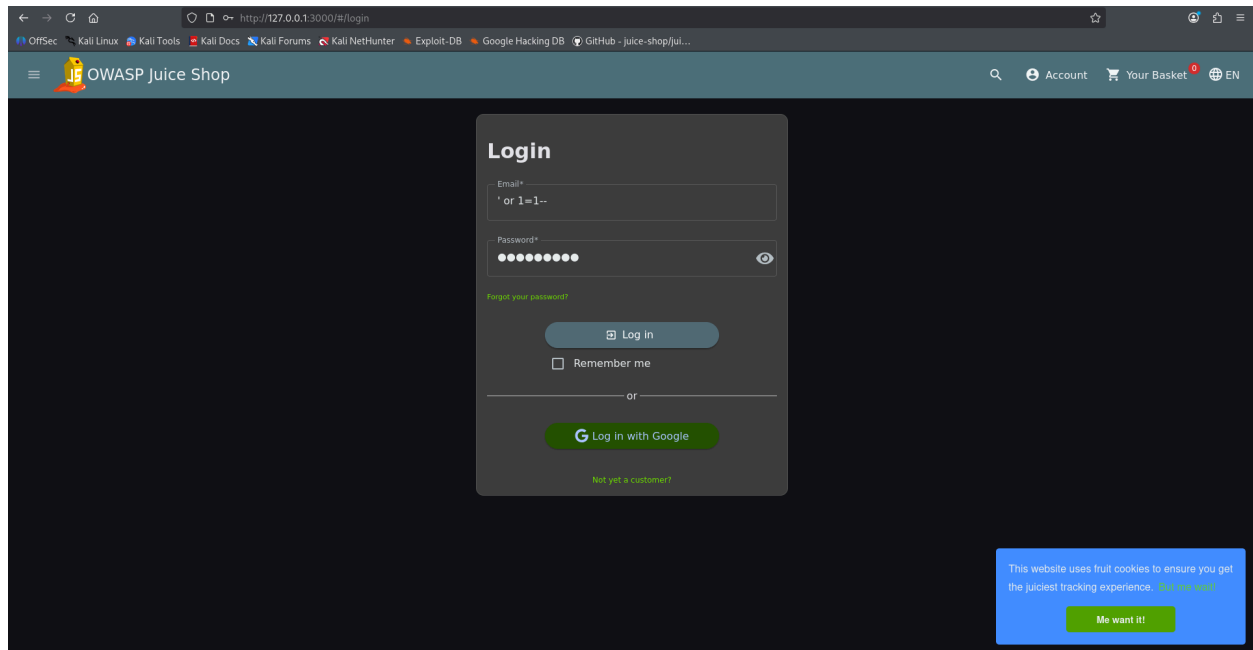
The web app does not allow create a new user with default password

3, Testing for weak lock out mechanism

The web application does not provide any lock out mechanism to protect account log in activities

4, Testing for Bypassing Authentication Schema

By using sql injection we can log in the user that appear first in the account table in SQL



By getting into the admin account we can easily retrieve user list from the admin dashboard

5, Testing for Vulnerable Remember Password

Based on the previous account log in, we inspect the cookies that stores on our browser, using the Base64 decoder , we can get the password in MD5 hash and other information related to the admin account

Simply enter your data then push the decode button.

i For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

< DECODE > Decodes your data into the area below.

 Copy to clipboard

Select a file to upload and process, then you can download the decoded result.

Free Password Hash Cracker

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin), QubesV3.1BackupDefaults

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

Back button show sensitive data:

[← Back](#)  OWASP Juice Shop

Data Erasure Request (Art. 17 GDPR)

We take data security, customer privacy, and legal compliance very serious. In accordance with GDPR we allow you to request complete erasure of your account and any associated data.

Request Data Erasure

Confirm Email Address

Answer

[x Delete User Data](#)

6, Testing for weak password policy

Password minimum requirements are 5 characters long and maxim are 40 characters long making brute-force are possible

User Registration

Password must be 5-40 characters long.

0/40



Show password advice

! This cannot be changed later!

+ Register

Already a customer?

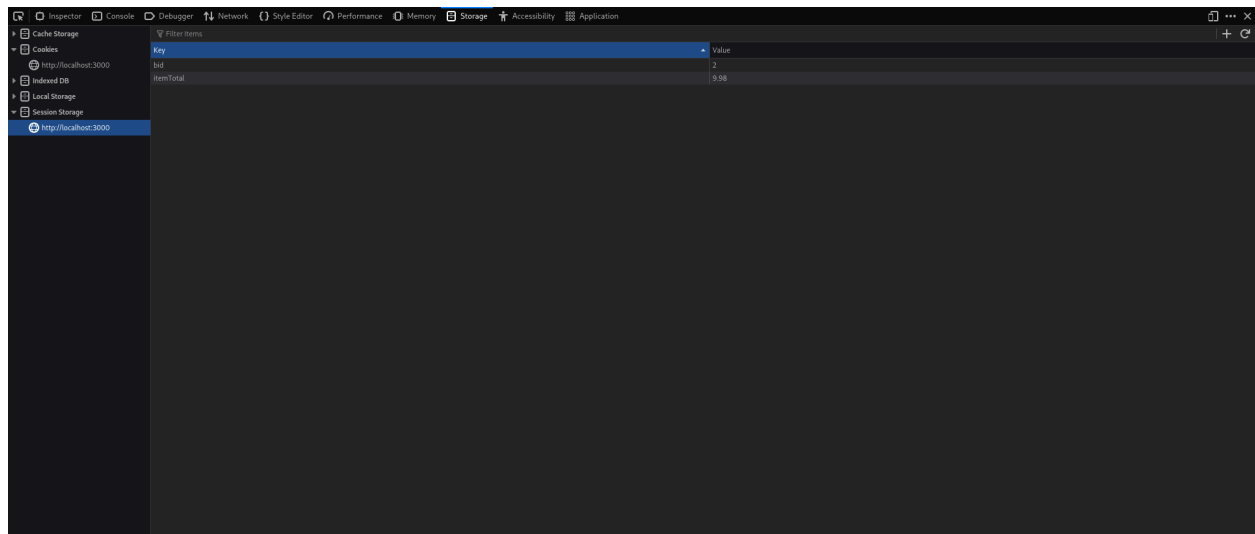
7, Testing for weak security question answer

The questions are all pre-generated that stick with aspect around life. Attacker can easily get the answer by perform social engineering

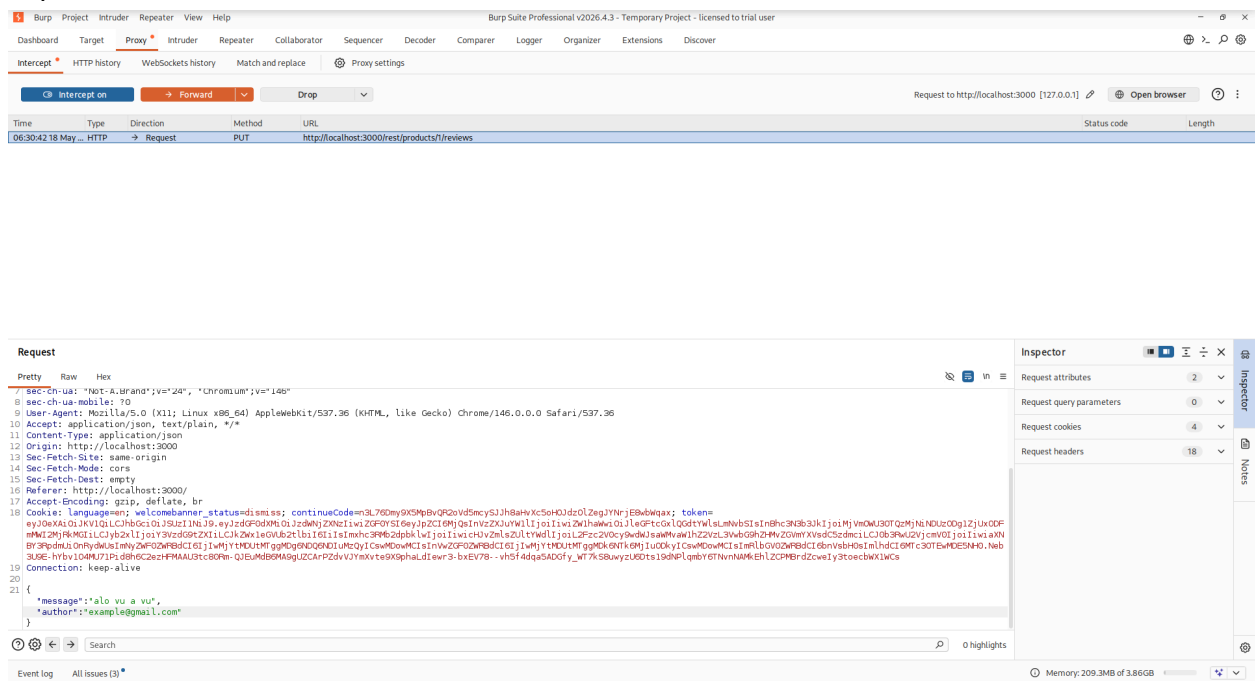
III, Authorization testing

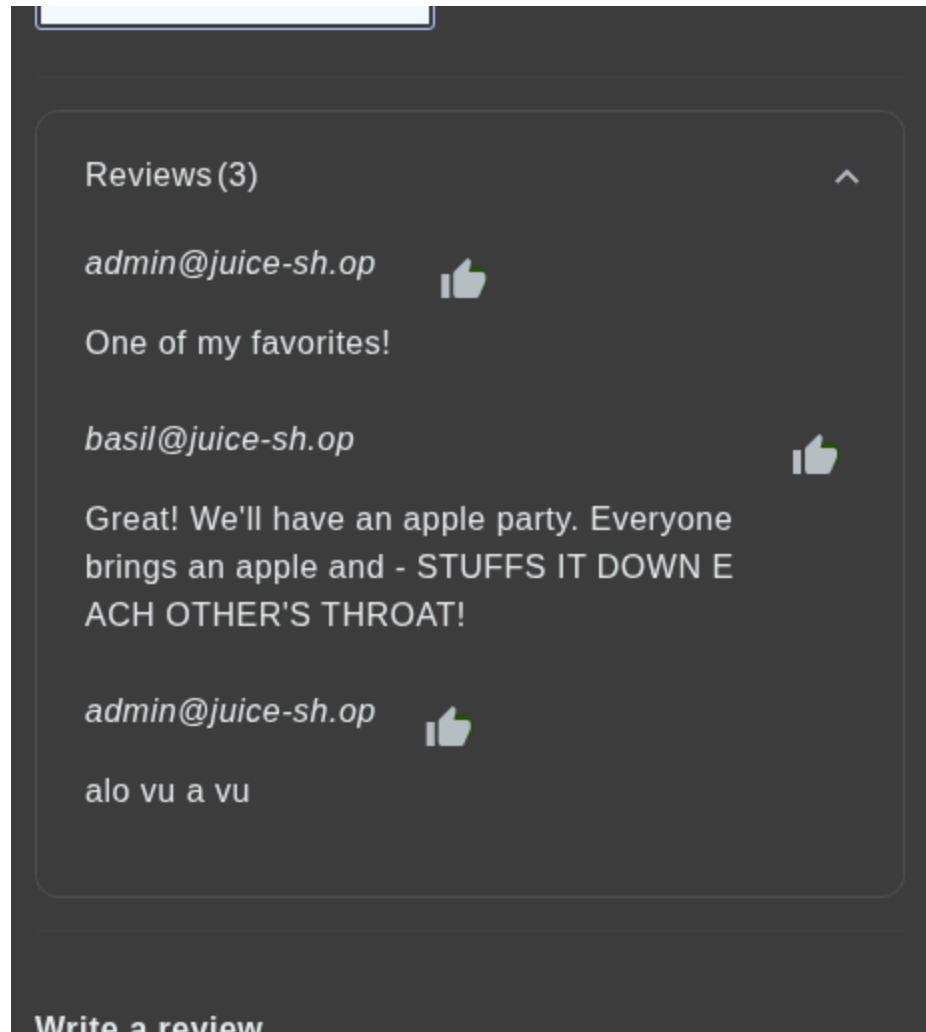
1, Testing for Bypassing Authorization schema

- User can view other basket by change the value bid in the session token:

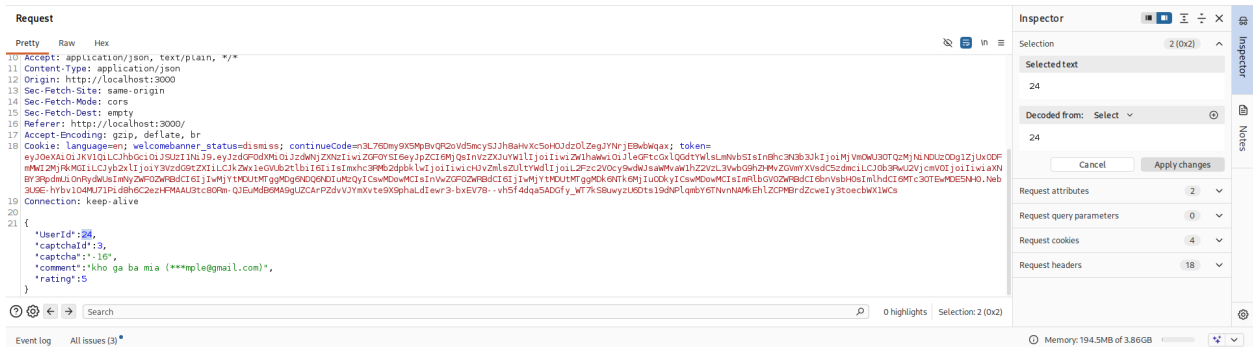
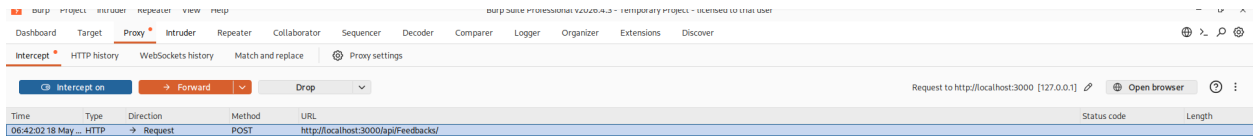


- User can change the author of a review for a production by change the author field in the PUT request

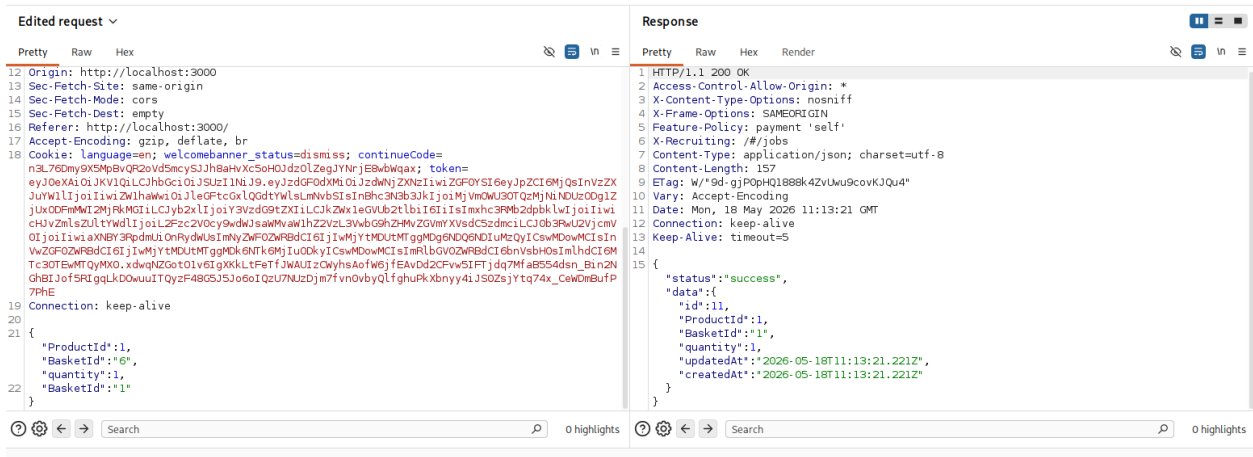




- By changing the Used ID field in the POST request we can be someone else doing the feedback



- user can add items to other user's Basket by edit the Basket ID in the POST request and using HTTP parameter pollution



Response

PrettyRawHexRender

1 HTTP/1.1 201 Created

2 Access-Control-Allow-Origin: *

3 X-Content-Type-Options: nosniff

4 X-Frame-Options: SAMEORIGIN

5 Feature-Policy: payment 'self'

6 X-Recruiting: /#/jobs

7 Location: /api/Feedbacks/10

8 Content-Type: application/json; charset=utf-8

9 Content-Length: 183

10 ETag: W/"b7-9z9obt6kpJ/oZJ/RjJVitgIuhc"

11 Vary: Accept-Encoding

12 Date: Mon, 18 May 2026 10:42:59 GMT

13 Connection: keep-alive

14 Keep-Alive: timeout=5

15

16 {

17 "status": "success",

18 "data": {

19 "id": 10,

20 "UserId": 1,

21 "comment": "kho ga ba mia (**mp@gmail.com)",

22 "rating": 5,

23 "updatedAt": "2026-05-18T10:42:59.664Z",

24 "createdAt": "2026-05-18T10:42:59.664Z"

25 }

26 }

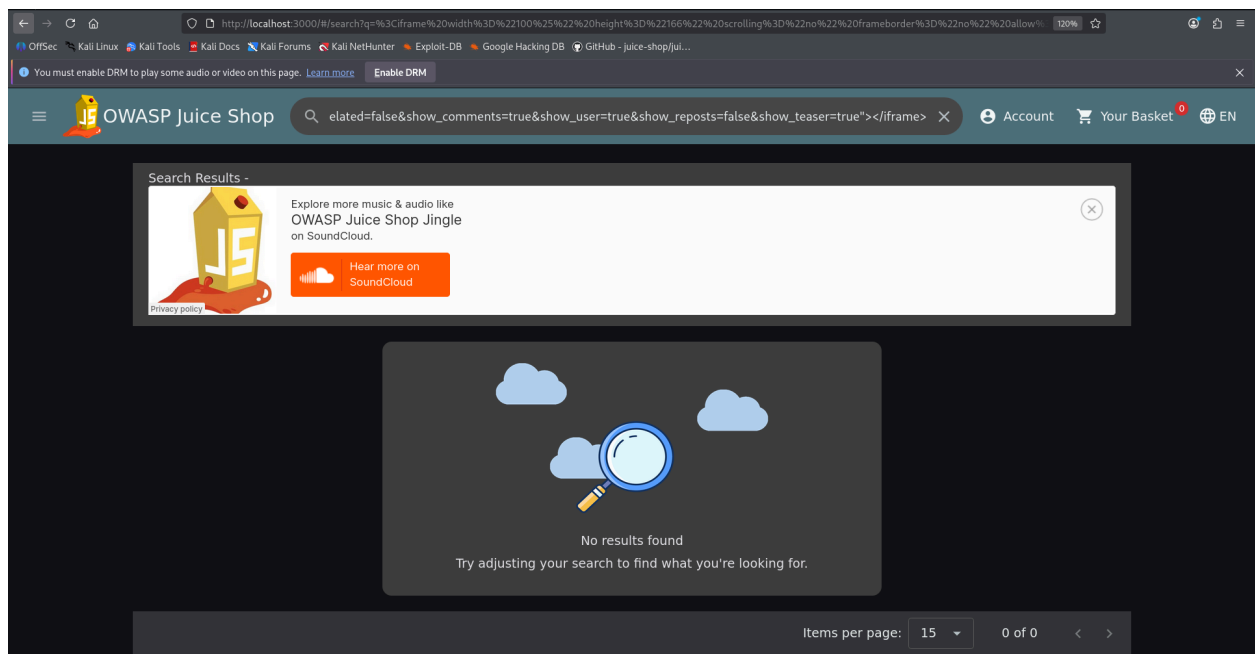
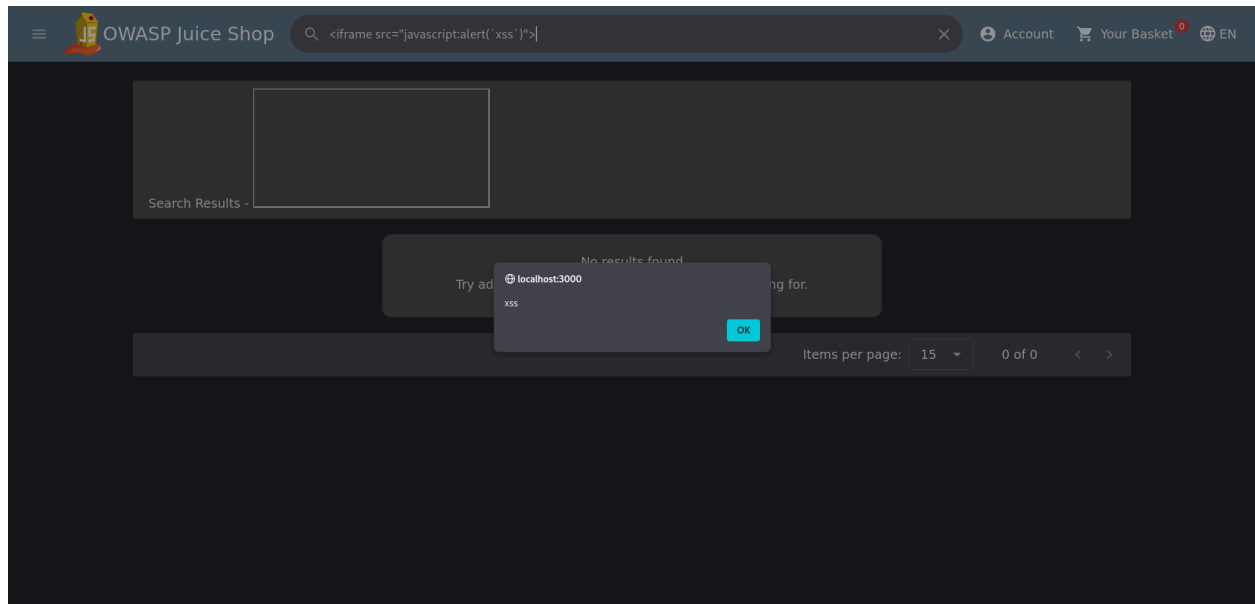
Search

0 highlights

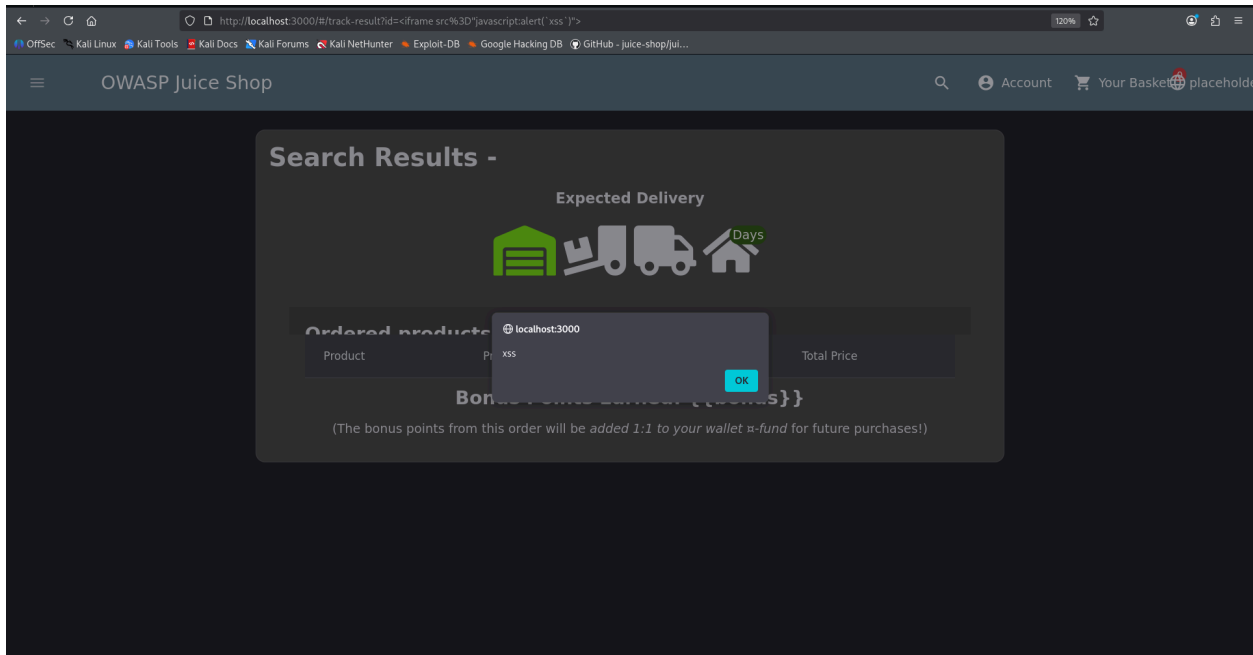
IV, INPUT VALIDATION TESTING

1, Testing for XSS

- The web application are vulnerable with DOM XSS and can be exploited through Search bar



- Web app have a Reflected XSS vulnerability



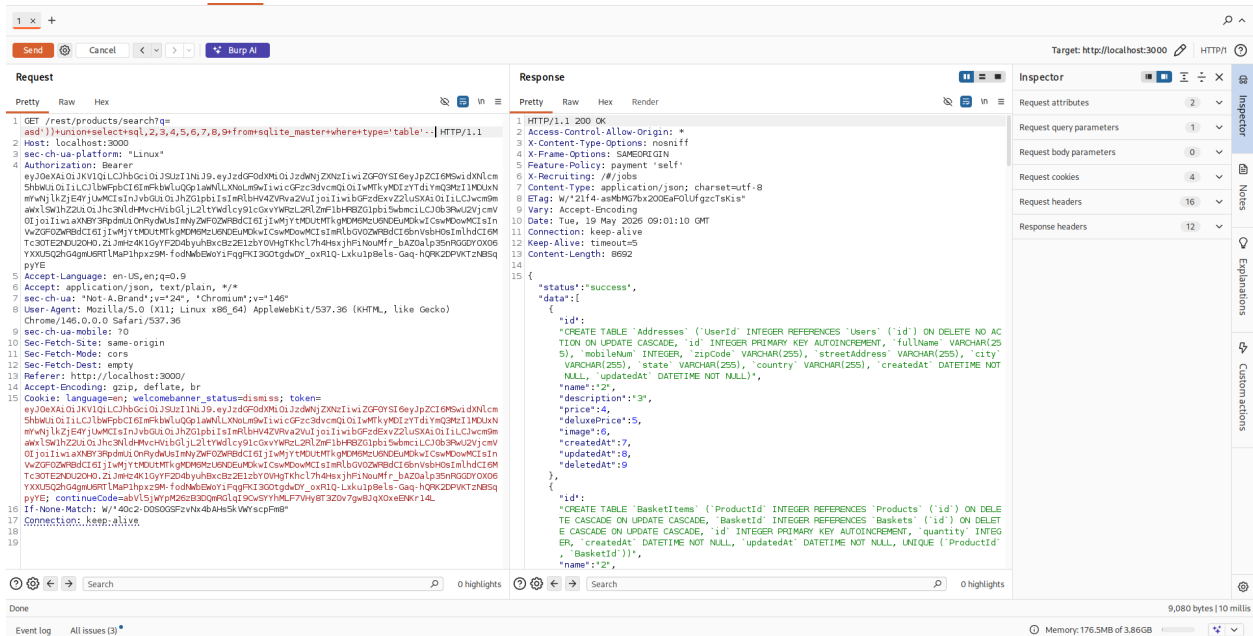
2, Testing for HTTP parameter solution

Mentioned above earlier

3, Testing for SQL injection

Mentioned above earlier

- We can extract DB schema using sql injection through API



- We can log in as a user that never existed

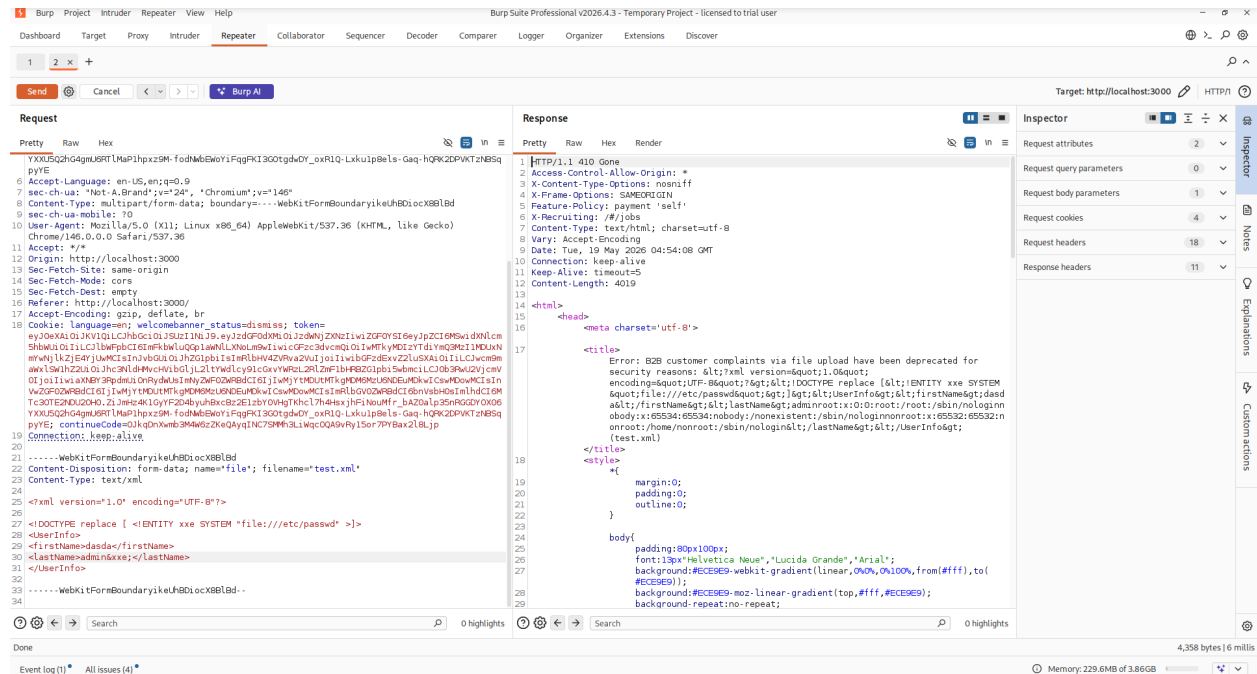
The screenshot shows the Burp Suite interface with a target set to http://localhost:3000. A POST request to /rest/user/login is shown in the Request tab. The response is a 200 OK status with a JSON body containing an authentication token and user details. The interface includes tabs for Request, Response, and Inspector, along with a search bar and various toolbars.

- User credentials can be extracted through API gateway

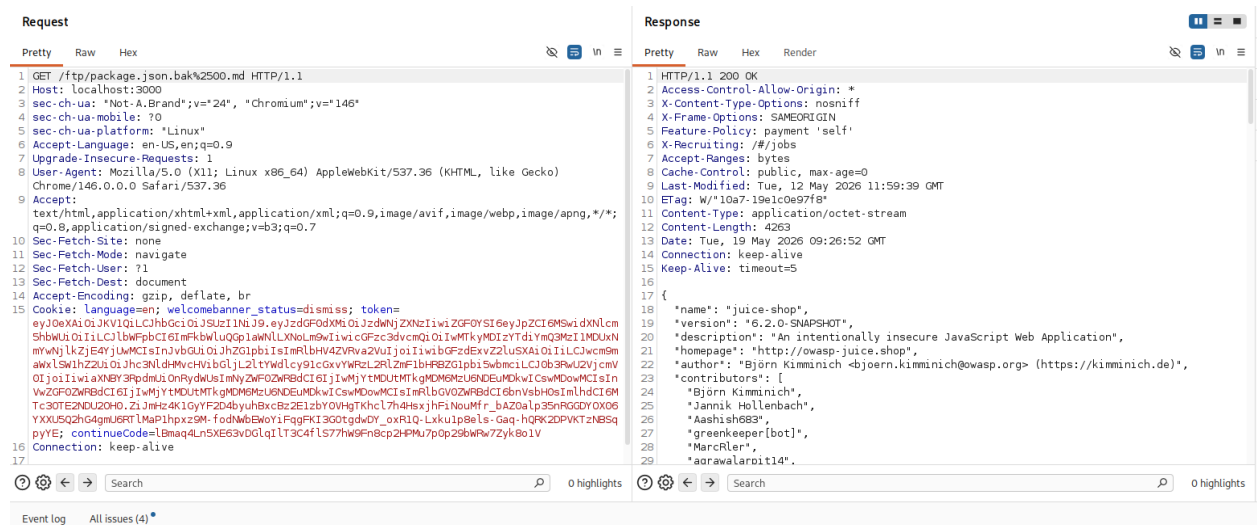
The screenshot shows the Burp Suite interface with a target set to http://localhost:3000. A GET request to /rest/products/search?q= is shown in the Request tab. The response is a 200 OK status with a JSON body containing product details. The interface includes tabs for Request, Response, and Inspector, along with a search bar and various toolbars.

4, Testing for XML injection

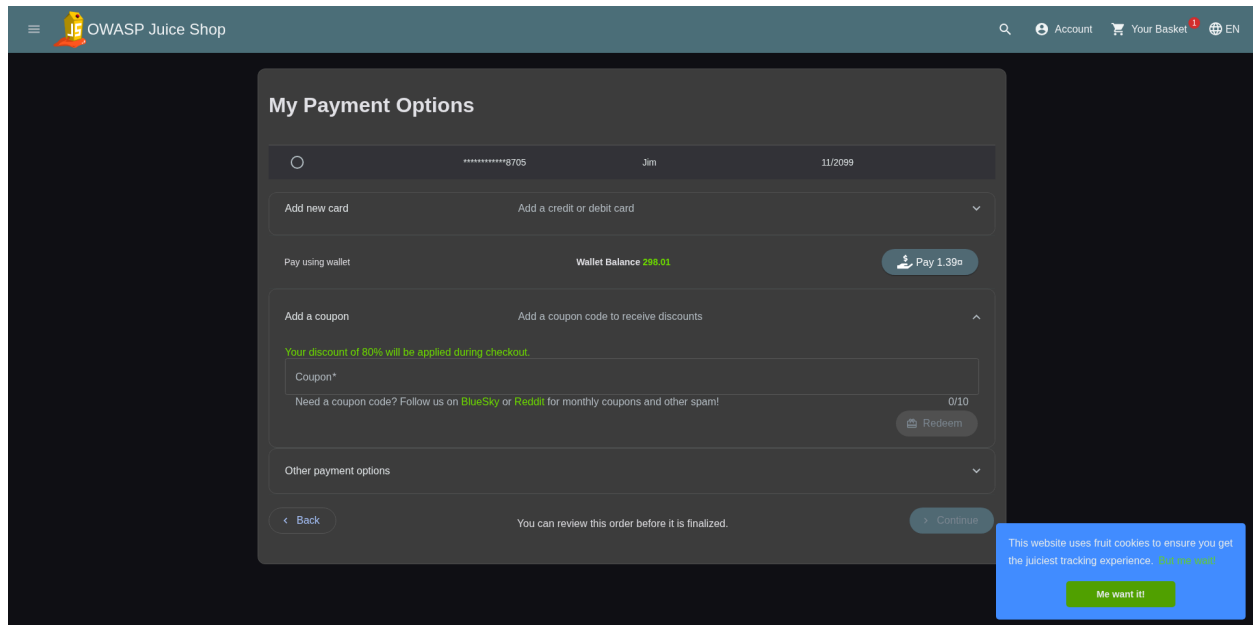
- Upload data that is not properly handled and XML file can be uploaded and inject



- URL can be inject by using Null byte



Web application are vulnerable to SSTi , which allow us to inject code to run on victim machine



REPORT FOR OWASP PEN TEST

I, Information gathering

- Hidden path are being exposed by variable in main java script file
- Web frontend engine and database software is revealed through error handling
- Web application expose ftp and showing a file directory that contains sensitive information

II, Authentication testing

- Web applications are using unencrypted channel HTTP instead of HTTPS
- Web application does not provide any lock out mechanism for account
- Authentication schema can be bypassed using SQL injection
- Cookies stored user credentials in MD5 Hash form , which can easily be exploited by using rainbow table
- Weak password policy
- Pre-Generated Security question can be easily exploited by doing Social Engineering

III, Authorization testing

- User can easily see other's basket by changing the "bid" value in the session token
- User can use PUT request on /rest to perform malicious request to change the data that stored in web server
- User can use PUT request on /api to perform malicious request to change the data that stored in web server

IV, Input Validation testing

- The web server are vulnerable to XSS through :
 - + Search bar
 - + Product review
- Web applications are vulnerable to HTTP parameter pollution through PUT request on API
- Web applications are vulnerable to SQL injection through:
 - + API

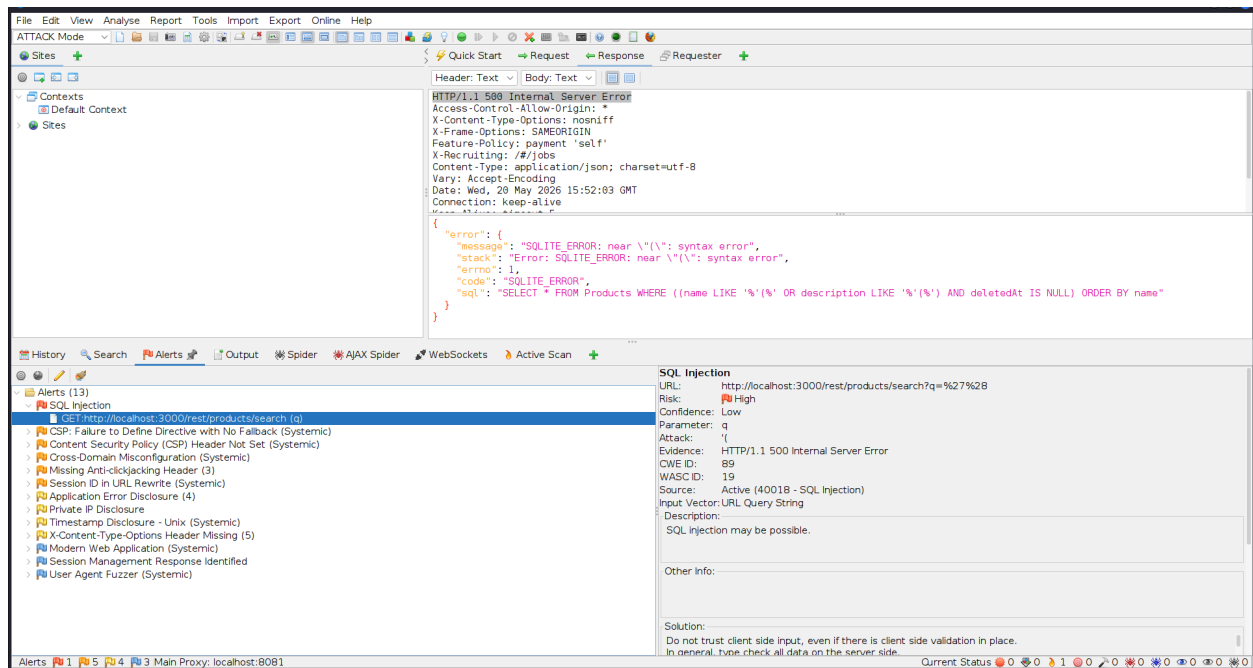
+ Authentication form

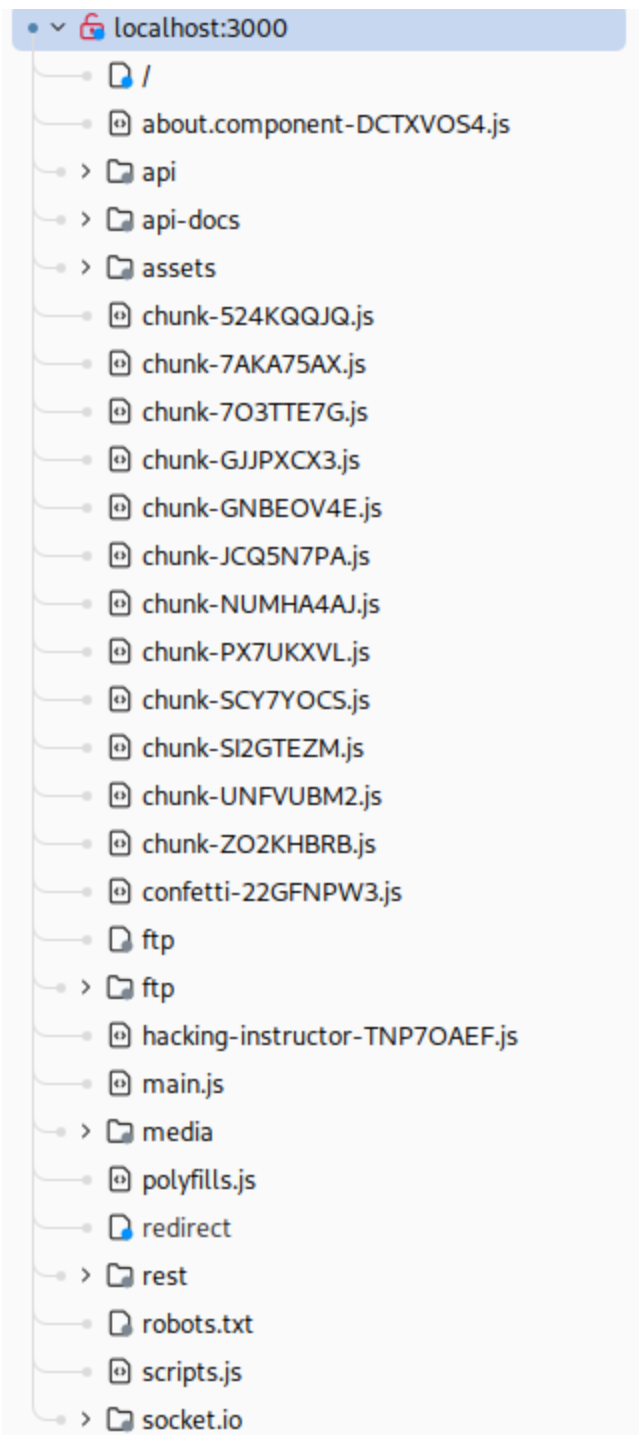
- Web applications are vulnerable to XML injection due to file upload function
- Web application can be injected through URL by using Null byte
- Web application are vulnerable to code injection through change username function

V, Testing business logic testing

- User can craft a request that add negative items into their basket
- User can forge a discount coupon by reverse engine the way a coupon is created

TESTING WITH AUTOMATED TOOLS





Tasks

New scan

New live task

⊙

⊙

Filter

Search

3. Crawl and audit of localhost:3000

Crawl and Audit - Deep

✓ Finished

Issues: 0 0 2 61

2. Live audit from Proxy (all traffic)

Audit checks - passive

Capturing

Issues: 0 0 0 0

1. Live passive crawl from Proxy (all traffic)

Add links. Add item itself, same domain and URLs in suite scope.

Capturing

3. Crawl and audit of localhost:3000

Rate this scan

Summary

Audit items

Issues

Event log

Logger

Audit log

Live crawl view

Most serious vulnerabilities found (live)

View all

Issue type	Host	Time
1 Open redirection (reflected)	http://localhost:3000	23:49:30 20 May 2026
1 Unencrypted communications	http://localhost:3000	23:44:58 20 May 2026
1 Cross-origin resource sharing	http://localhost:3000	23:48:53 20 May 2026
1 Cross-origin resource sharing	http://localhost:3000	23:52:46 20 May 2026
1 Cross-origin resource sharing	http://localhost:3000	23:54:40 20 May 2026
1 Cross-origin resource sharing	http://localhost:3000	23:55:14 20 May 2026
1 Cross-origin resource sharing	http://localhost:3000	23:56:24 20 May 2026
1 Cross-origin resource sharing	http://localhost:3000	23:56:58 20 May 2026
1 Cross-origin resource sharing	http://localhost:3000	23:57:45 20 May 2026
1 Cross-origin resource sharing	http://localhost:3000	23:58:31 20 May 2026
1 Cross-origin resource sharing	http://localhost:3000	23:58:35 20 May 2026
1 Cross-origin resource sharing	http://localhost:3000	00:00:09 21 May 2026
1 Cross-origin resource sharing	http://localhost:3000	00:00:11 21 May 2026
1 Cross-origin resource sharing	http://localhost:3000	00:00:15 21 May 2026
1 Cross-origin resource sharing	http://localhost:3000	00:00:22 21 May 2026
1 Cross-origin resource sharing	http://localhost:3000	00:00:27 21 May 2026
1 Cross-origin resource sharing	http://localhost:3000	00:00:35 21 May 2026
1 Cross-origin resource sharing	http://localhost:3000	00:00:40 21 May 2026
1 Cross-origin resource sharing	http://localhost:3000	00:00:46 21 May 2026
1 Cross-origin resource sharing	http://localhost:3000	00:01:25 21 May 2026
1 Cross-origin resource sharing	http://localhost:3000	00:01:49 21 May 2026
1 Cross-origin resource sharing	http://localhost:3000	00:02:07 21 May 2026
1 Cross-origin resource sharing: arbitrary origin tru...	http://localhost:3000	23:48:53 20 May 2026
1 Cross-origin resource sharing: arbitrary origin tru...	http://localhost:3000	23:52:46 20 May 2026
1 Cross-origin resource sharing: arbitrary origin tru...	http://localhost:3000	23:54:40 20 May 2026
1 Cross-origin resource sharing: arbitrary origin tru...	http://localhost:3000	23:55:14 20 May 2026
1 Cross-origin resource sharing: arbitrary origin tru...	http://localhost:3000	23:56:24 20 May 2026
1 Cross-origin resource sharing: arbitrary origin tru...	http://localhost:3000	23:56:58 20 May 2026
1 Cross-origin resource sharing: arbitrary origin tru...	http://localhost:3000	23:57:45 20 May 2026
1 Cross-origin resource sharing: arbitrary origin tru...	http://localhost:3000	23:58:31 20 May 2026

Task configuration

View configuration

Task type: Crawl & audit

Scope: localhost:3000

Configuration: Crawl and Audit - Deep

Task progress

Total audit items:	75	Discovered locations:	42
Audit items in progress:	0	Discovery actions pending:	0
Audit items completed:	75	Requests:	61273
Network errors:	2575	Requests per second:	0

Task log

> Auditing "http://localhost:3000/" for ASP Debugging Enabled

> Auditing "http://localhost:3000/" for Hidden HTTP2

> Auditing "http://localhost:3000/" for OpenAPI Definition Found

> Auditing "http://localhost:3000/" for JWK Disclosed

> Auditing "http://localhost:3000/" for React2Shell RCE

> Auditing "http://localhost:3000/" for GraphQL Endpoint Discovered

> Auditing "http://localhost:3000/" for Flash Crossdomain Policy

> Auditing "http://localhost:3000/" for Robots TXT and TLS Config

> Auditing "http://localhost:3000/" for HTTP PUT Enabled

> Auditing "http://localhost:3000/" for Silverlight Crossdomain Policy

> Auditing "http://localhost:3000/" for GraphQL Endpoint Discovered

> Auditing "http://localhost:3000/" for HTTP PUT Enabled

> Auditing "http://localhost:3000/" for JWK Disclosed

> Auditing "http://localhost:3000/" for React2Shell RCE

> Auditing "http://localhost:3000/" for Robots TXT and TLS Config

> Auditing "http://localhost:3000/" for ASP Debugging Enabled

> Auditing "http://localhost:3000/" for Flash Crossdomain Policy

> Auditing "http://localhost:3000/" for ASP Tracing App Level

> Auditing "http://localhost:3000/" for Collaborator Everywhere

> Auditing "http://localhost:3000/" for OpenAPI Definition Found

> Auditing "http://localhost:3000/" for Trace Method

Event log (7)

All issues (63)

Memory: 458.2MB of 3.86GB

REPORT FOR USING ZAP AND BURP SITE AUTOMATED SCAN TOOL

- Vulnerability detected : SQL injection
- Detect most of the web app path.

COMPARE:

- Automated scans are good at Crawling and Spidering but really bad at spot vulnerabilities that requires context
- OWASP WSTG is excellent at guiding tester to spot vulnerabilities but very complex and slow since a web app can have hundreds of attack vector